



Chapitre 6: Les tableaux

Babacar DIOP

Adresse Mail: diopbabacar4888@gmail.com

Téléphone: 777812040

Introduction

- ❖ Jusqu'à présent, une variable en C ne peut contenir qu'**une seule donnée à la fois**.
- ❖ Lorsqu'on doit gérer plusieurs données, utiliser plusieurs variables devient vite compliqué et peu pratique.
- ❖ Le langage C offre des structures de données permettant de regrouper plusieurs valeurs dans une seule entité.
- ❖ Grâce à ces structures complexes, il devient possible d'accéder et de manipuler facilement un ensemble de données via une seule variable.
- ❖ Un tableau est une **variable qui regroupe plusieurs données du même type, stockées les unes à la suite des autres en mémoire**. Chaque élément occupe un espace de taille fixe, déterminé par le type de données (ex. : int, char, float, etc.).
- ❖ Les éléments d'un tableau peuvent être :
 - des types simples (int, char, float, etc.)
 - des pointeurs (adresses en mémoire)
 - d'autres tableaux
 - ou des structures (groupes de données hétérogènes)
- ❖ Lorsqu'un tableau contient uniquement des données simples (comme des nombres ou des caractères), on l'appelle **tableau unidimensionnel ou vecteur** ; s'il contient d'autres tableaux, on parle alors de **tableau multidimensionnel, aussi appelé matrice ou table**.



I. Tableau unidimensionnel

1. Déclaration

- ❖ Un tableau unidimensionnel est un tableau qui contient des éléments simples (des éléments qui ne sont pas des tableaux). Un tableau unidimensionnel est donc une suite de « cases » de même taille contenant des éléments d'un type donné (de la longueur de la case en quelque sorte)..
- ❖ Un tableau monodimensionnel est donc une suite de « cases » de même taille contenant des éléments d'un type donné (de la longueur de la case en quelque sorte).

Exemple: Un tableau contenant des entiers

int	int	int	...	int	int	int
-----	-----	-----	-----	-----	-----	-----

- ❖ Syntaxe ***Type Nom_du_tableau [Nombre d'éléments]***

- **Type** définit le type d'élément que contient le tableau
- **Nom_du_tableau** est le nom que l'on décide de donner au tableau, le nom du tableau suit les mêmes règles qu'un nom de variable
- **Nombre d'éléments** est un nombre entier qui détermine le nombre de cases que le tableau doit comporter

- ❖ Exemple : Déclaration d'un tableau qui doit contenir 8 éléments de type char ***char Tableau [8];***

- ❖ Un tableau en C ayant un nombre fixe d'éléments d'un type donné, sa taille en mémoire est connue dès sa déclaration. Pour connaître cette taille (en octets), on peut soit la calculer manuellement en multipliant le nombre d'éléments par la taille d'un élément, soit utiliser l'opérateur **sizeof()**, qui permet d'obtenir automatiquement la taille d'un élément passé en argument.

I. Tableau unidimensionnel

2. Manipulation

❖ Chaque élément d'un tableau, identifié par le nom du tableau suivi de son indice, se comporte comme une variable simple : on peut donc l'utiliser dans des opérations, le modifier, ou lui affecter une valeur comme à n'importe quelle variable.

❖ Exemple:

```
❑ int notes[10];      // Déclaration d'un tableau pour stocker 10 notes
❑ notes[7] = 6;       // Affectation de la valeur 6 à la huitième note
❑ notes[9] = notes[0] + notes[1];      //Affectation de la dixième note la somme des deux premières
```

❖ Quand on déclare un tableau, ses éléments ne contiennent pas de valeurs définies. Il faut donc les initialiser, c'est-à-dire leur donner une valeur.

❖ Initialisation élément par élément

```
notes[0] = notes[1] = notes[2] = 0;
```

méthode est longue et peu efficace si le tableau est grand

❖ Initialisation avec une **boucle for**

```
int notes[10];
int i;
for (i = 0; i < 10; i++) {
    notes[i] = 0;
}
```

Cette méthode est utile si tous les éléments doivent être initialisés à la même valeur ou à une valeur liée à l'indice.

I. Tableau unidimensionnel

2. Manipulation (suite)

- ❖ Initialisation directe à la déclaration: Si l'on connaît déjà les valeurs, on peut les spécifier directement lors de la définition :

```
int notes[10] = {1, 2, 6, 5, 2, 1, 9, 8, 1, 5};
```

Quelques règles à retenir :

Le nombre de valeurs ne doit pas dépasser la taille du tableau.

Les valeurs doivent être des valeurs (pas de variables).

Si on fournit moins de valeurs, les cases restantes sont remplies avec 0.

Exemple pour tout mettre à zéro :

```
int notes[10] = {0};
```

- ❖ Utilisation de constantes pour la taille: Il est recommandé d'utiliser des **constantes** pour définir la taille d'un tableau.

Exemple:

```
#define NB_NOTES 10  
int notes[NB_NOTES];  
for (int i = 0; i < NB_NOTES; i++) {  
    notes[i] = 0;  
}
```

méthode avec moins d'erreurs liées aux dépassements de tableau, facile à modifier.

III. Tableau multidimensionnel

1. Déclaration

- ❖ Les tableaux multidimensionnels sont des tableaux qui contiennent des tableaux.
- ❖ Par exemple le tableau bidimensionnel (3 lignes, 4 colonnes) suivant, est en fait un tableau comportant 3 éléments, chacun d'entre eux étant un tableau de 4 éléments :

est stocké en mémoire de la manière suivante



- ❖ Déclaration: **Type Nom_du_tableau [a1][a2][a3] ... [aN]**

- ☐ Chaque élément entre crochets désigne le nombre d'éléments dans chaque dimension
- ☐ Le nombre de dimensions n'est pas limité

- ❖ Exemple: Un tableau d'entiers positifs à deux dimensions (3 lignes, 4 colonnes) : **int Tableau [3][4];**

Tableau[0][0]	Tableau[0][1]	Tableau[0][2]	Tableau[0][3]
Tableau[1][0]	Tableau[1][1]	Tableau[1][2]	Tableau[1][3]
Tableau[2][0]	Tableau[2][1]	Tableau[2][2]	Tableau[2][3]

III. Tableau multidimensionnel

2. Manipulation

❖ L'initialisation d'un tableau multidimensionnel se fait de la même manière que pour un tableau simple, mais avec plusieurs indices.

❖ Initialisation manuelle

```
tableau[0][0] = 2;  
tableau[0][1] = 3;
```

❖ Initialisation avec boucles imbriquées: Chaque dimension du tableau est représentée par une boucle.

❖ Initialisation directe à la déclaration

```
int tableau[2][3] = {1, 2, 3, 4, 5, 6};
```

Les valeurs sont placées dans l'ordre : d'abord [0][0], puis [0][1], etc. jusqu'à [1][2].

```
int i, j;  
for (i = 0; i < 3; i++) {  
    for (j = 0; j < 4; j++) {  
        tableau[i][j] = 0;  
    }  
}
```

❖ Initialisation avec **memset**: Si l'on veut mettre toutes les cases à 0 rapidement, on peut utiliser la fonction memset

```
#include <string.h>
```

```
int tableau[3][4];  
memset(tableau, 0, sizeof(tableau));
```

Exercices d'Application

Écrire un programme qui permet de remplir un tableau de 50 étudiants. Le programme détermine et affiche les informations de l'étudiant le plus âgé et celles du moins âgé. Un étudiant est caractérisé par son matricule, son nom, son prénom et son année de naissance.

Ecrire un programme qui remplit et affiche une matrice carrée de 10000 produits. Le programme détermine et affiche le pourcentage de présence des produits dont leur libellé contient le mot « lait », le montant total des produits se trouvant sur la diagonale principale et le nombre de produits se trouvant sur la diagonale secondaire.

PRODUIT (code(chaine(15), libelle(chaine(75)), prix(entier), quantite(réel))